**National University of Computer & Emerging Sciences**

Chiniot-Faisalabad (CFD) Campus

# CL2005 – Database Systems Lab

## Semester Project Manual

*Spring 2026*


**Lab Instructor:** Hassan Ahmed • **Email:** hassan.ahmed@nu.edu.pk

# Contents

# 1 Project Overview

The Database Lab Semester Project is a **comprehensive, team-based assessment** that integrates all core competencies developed across the lab course. Each group of two students will design, develop, and deploy a fully functional database-driven application that demonstrates mastery of relational database design, SQL programming, PL/SQL procedural development, and graphical user interface (GUI) implementation.

**Objective**

To build a real-world database application from scratch — covering requirements analysis, ER/EER modeling, schema implementation, DML/DDL operations, PL/SQL programming, and a user-facing GUI dashboard — thereby satisfying all Lab Learning Outcomes (LLO-1 through LLO-4).

## 1.1 LLO–Project Alignment

| LLO | Statement | Project Phase |
|---|---|---|
| **LLO-1** | File handling, MS Access, SQL Server tools | Phase 1 + Phase 2 |
| **LLO-2** | Relational DB design & structured query processing | Phase 2 + Phase 3 |
| **LLO-3** | PL/SQL procedural programming | Phase 4 |
| **LLO-4** | Individual and team professionalism | All Phases |

Table 1: Mapping of Lab Learning Outcomes to Project Phases

# 2 Group Formation & Topic Selection

## 2.1 Group Policy

- Each group consists of **exactly 2 members**. No solo or larger groups are permitted.
- Groups must be finalized and submitted to the instructor by **Week 13**.
- Each member must contribute to **all phases** of the project. Role-based division is allowed, but both members must demonstrate understanding during the viva.
- Any form of plagiarism — between groups or from online sources (GenAI) — will result in **zero marks** for all involved parties.

## 2.2 Project Domain Selection

Groups must choose a project domain from the approved list below, or propose a custom domain subject to instructor approval.

| # | Domain | Complexity |
|---|---|---|
| 1 | Hospital Management System | High |
| 2 | University Enrollment System | High |
| 3 | Library Management System | Medium |
| 4 | Online Retail / E-Commerce System | High |
| 5 | Hotel Reservation System | Medium |
| 6 | Airline Reservation System | High |
| 7 | Bank Account Management System | High |
| 8 | Inventory & Warehouse Management | Medium |
| 9 | Restaurant Order Management | Medium |
| 10 | Human Resource Management System | High |
| 11 | School Fee Management System | Medium |
| 12 | Custom Domain (with approval) | Variable |

Table 2: Approved Project Domains

**⚠ Important Note**

Two groups **cannot** select the same domain. Domains are allocated on a first-come, first-served basis after instructor confirmation.

# 3 Project Phases & Deliverables

The project is divided into **five phases**. Each phase has specific quantitative requirements that must be met.

## 3.1 Phase 1 — Requirements Analysis & Documentation [LLO-1][LLO-4]

### 3.1.1 Deliverable

A written **Software Requirements Specification (SRS)** document and a **Data Dictionary**.

### 3.1.2 Minimum Quantitative Requirements

**Phase 1 – Minimum Requirements**

- ✱ **Entities:** Identify and document a minimum of **8 distinct entities** relevant to your domain.
- ✱ **Attributes per Entity:** Each entity must have at minimum **5 attributes** (including primary key).
- ✱ **Functional Requirements:** List at least **10 functional requirements** (use cases).
- ✱ **Data Dictionary:** Include a data dictionary table for every entity with column name, data type, constraints, and description.
- ✚ **Non-Functional Requirements:** At least 3 non-functional requirements (performance, security, scalability).

### 3.1.3 Data Dictionary Format

Each entity's data dictionary must follow this format:

| Column Name | Data Type | Size | Constraints | Description |
|---|---|---|---|---|
| **patient_id** | NUMBER | 10 | PK, NOT NULL | Unique identifier for each patient |
| **full_name** | VARCHAR2 | 100 | NOT NULL | Full legal name of the patient |
| **dob** | DATE | – | NOT NULL | Date of birth |
| **gender** | CHAR | 1 | CHECK(M/F/O) | Gender indicator |
| **contact_no** | VARCHAR2 | 15 | UNIQUE | Mobile or landline number |

Table 3: Example Data Dictionary Entry (Patient Entity)

## 3.2 Phase 2 — Database Design & Schema Implementation [LLO-1][LLO-2]

### 3.2.1 Sub-Phase 2A: ER and EER Diagrams

**ER/EER Requirements**

- ✱ Draw an **Entity-Relationship Diagram (ERD)** covering all entities.
- ✱ Draw an **Enhanced ERD (EERD)** showing at minimum:
  - 1 specialization/generalization hierarchy (with at least 2 subtypes)
  - At least 1 aggregation or composition relationship
- ✱ **Minimum 8 entities** and **minimum 10 relationships** in the ERD.
- ✱ Clearly label all cardinalities: at least **2 one-to-many**, **1 many-to-many**, and **1 one-to-one** relationship.
- ✱ Distinguish weak entities where applicable (at least **1 weak entity** required).
- ✱ Tools: Use any standard tool (draw.io, Lucidchart, MS Visio, or ERDPlus).

### 3.2.2 Sub-Phase 2B: Relational Schema and DDL

**Schema & DDL Requirements**

- ✱ **Minimum 8 tables** created using DDL `CREATE TABLE` statements.
- ✱ Each table must include:
  - Primary Key constraint
  - Appropriate data types (NUMBER, VARCHAR2, DATE, CHAR, etc.)
  - At least one `NOT NULL` constraint
- ✱ **Minimum 6 foreign key** relationships enforced with `REFERENCES`.
- ✱ Use at least **3 CHECK constraints** across the schema.
- ✱ Use at least **2 UNIQUE constraints**.
- ✱ Apply **ON DELETE CASCADE** or **ON DELETE SET NULL** on at least 2 foreign keys.
- ✱ Normalize all tables to at least **Third Normal Form (3NF)** — justify in documentation.
- ✱ Create at least **2 indexes** on frequently queried columns.
- ✱ Create at least **2 views** for common query patterns.

### 3.2.3 Sub-Phase 2C: Data Population

**Data Population Requirements**

- ✱ Insert a minimum of **20 rows per major table** (tables with 4+ columns).
- ✱ Insert a minimum of **10 rows per lookup/reference table**.
- ✱ Data must be realistic and domain-consistent (not placeholder values like "abc", "123").
- ✱ Use `INSERT INTO` DML statements — these must be included in the submission script.

### 3.2.4 Sub-Phase 2D: DML Queries

**Query Requirements**

Groups must implement and demonstrate the following queries:

- ✱ **5 SELECT queries** with `WHERE` clauses (simple filters).
- ✱ **3 aggregate queries** using `GROUP BY` and at least 3 aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`).
- ✱ **3 subqueries** (at least 1 correlated and 1 nested).
- ✱ **4 JOIN queries**: at least 1 INNER JOIN, 1 LEFT OUTER JOIN, 1 multi-table JOIN (3+ tables).
- ✱ **2 UPDATE** and **2 DELETE** statements with appropriate `WHERE` conditions.
- ✱ **1 DCL** demonstration: `GRANT` and `REVOKE` on at least one object.

## 3.3 Phase 3 — GUI Application Development [LLO-1][LLO-4]

### 3.3.1 Technology Choices

Groups may use any of the following:

- **C# Windows Forms** with Oracle or SQL Server connectivity
- **C# WPF** with database backend
- **Java Swing/JavaFX** with JDBC
- **Python (Tkinter / PyQt / Kivy)** with `cx_Oracle` or `pyodbc`
- **Web (HTML/CSS/JS + Node.js / PHP)** with database connectivity

### 3.3.2 Minimum GUI Requirements

**GUI Functional Requirements**

- ✱ **Login / Authentication Screen:**
  - Username and password fields with validation.
  - At least 2 user roles (e.g., Admin, User/Viewer).
  - Role-based menu/feature visibility.
- ✱ **CRUD Forms (minimum 3 entities):**
  - Each form must support: Add, Update, Delete, and Search.
  - Input validation on all fields (empty checks, data type, format).
  - Dropdown/ComboBox for FK references (no manual ID entry for FK columns).
- ✱ **Search & Filter:**
  - At least 1 form with multi-criteria search (filter by 2+ fields simultaneously).
- ✱ **DataGrid / Table Display:**
  - All records must be displayed in a grid or table view.
  - Grid must refresh automatically after add/update/delete.
- ✱ **Report Generation:**
  - Generate at least **2 printable or exportable reports** (PDF/Excel/CSV).
  - Reports must include title, date/time, and summary aggregates.
- ✱ **Navigation:** A main menu or sidebar linking to all modules.
- ✚ **Bonus:** Pagination for large datasets; date pickers for date fields.

## 3.4 Phase 4 — PL/SQL Implementation [LLO-3]

This phase is the most technically demanding and carries significant weight. All PL/SQL components must be implemented in Oracle (using Oracle SQL Developer or SQL*Plus).

**PL/SQL Minimum Requirements**

**4.1 Stored Procedures (minimum 3)**
- At least 1 procedure with `IN` and `OUT` parameters.
- At least 1 procedure using exception handling (`WHEN OTHERS`, named exceptions).
- At least 1 procedure calling another procedure (nesting).

**4.2 Functions (minimum 2)**
- Each function must return a computed value (not a simple SELECT).
- At least 1 function used inside a SQL query (i.e., called from SELECT).

**4.3 Triggers (minimum 3)**
- At least 1 **BEFORE INSERT** trigger (e.g., auto-generate ID or set default values).
- At least 1 **AFTER UPDATE** trigger (e.g., audit log, cascade update).
- At least 1 **AFTER DELETE** trigger (e.g., archival or logging).
- Triggers must perform meaningful business logic — not trivial `NULL` assignments.

**4.4 Cursors (minimum 2)**
- At least 1 **explicit cursor** with `OPEN`, `FETCH`, `CLOSE`.
- At least 1 **cursor with parameters**.
- Demonstrate use inside a loop to process multiple rows.

**4.5 Package (minimum 1)**
- Create 1 package containing at minimum:
  - 2 procedures
  - 1 function
  - 1 package-level variable or constant
- Demonstrate calling package members from an anonymous block.

**4.6 Anonymous PL/SQL Blocks (minimum 2)**
- At least 1 block demonstrating: `IF-ELSIF-ELSE`, loop (`FOR`/`WHILE`), and exception.
- At least 1 block that calls a stored procedure and prints output using `DBMS_OUTPUT`.

## 3.5 Phase 5 — Database Dashboard [LLO-1][LLO-2][LLO-4]

A dedicated **Dashboard / Analytics Screen** must be implemented either within the GUI application or as a standalone HTML/web report.

**Dashboard Requirements**

- ✱ **Minimum 4 KPI Cards** showing live aggregate data (e.g., Total Patients, Revenue This Month, Active Orders, Pending Requests).
- ✱ **Minimum 2 Charts/Graphs** (Bar Chart, Pie Chart, Line Chart, etc.) populated directly from database queries.
- ✱ **Minimum 1 Summary Table** showing grouped/aggregated data (e.g., top 10 records, monthly summaries).
- ✱ **Real-Time Refresh:** Dashboard data must reload from the database on every visit or on button click (no hardcoded values).
- ✚ **Bonus:** Date range filter for dashboard metrics.
- ✚ **Bonus:** Export dashboard to PDF or image.

# 4 Technical Standards & Coding Guidelines

## 4.1 Naming Conventions

| Object | Convention | Example |
|---|---|---|
| Table | Singular noun, UPPER_CASE | `PATIENT`, `DOCTOR` |
| Column | lower_case with underscores | `patient_id`, `full_name` |
| Primary Key | `tablename_id` | `patient_id` |
| Foreign Key | `ref_tablename_id` | `doctor_id` |
| Procedure | `proc_verb_noun` | `proc_add_patient` |
| Function | `fn_noun` | `fn_calculate_bill` |
| Trigger | `trg_timing_table_action` | `trg_before_patient_ins` |
| Package | `pkg_domain` | `pkg_billing` |
| View | `vw_description` | `vw_active_doctors` |
| Index | `idx_table_column` | `idx_patient_name` |

Table 4: Database Object Naming Conventions

## 4.2 SQL Script Structure

Every SQL/PL/SQL script file must:

- Begin with a header comment block containing: Group name, members, date, and script purpose.
- Use `-- Section --` comments to separate logical parts.
- Include `COMMIT;` after DML batches.
- Include `SET SERVEROUTPUT ON;` before anonymous blocks.

```
-- ============================================================
-- Project  : Hospital Management System
-- Script   : 02_ddl_schema.sql
-- Group    : Group 5
-- Members  : Ali Hassan (24F-1234), Sara Khan (24F-5678)
-- Date     : 2025-04-01
-- Purpose  : DDL - Create all tables with constraints
-- ============================================================

SET SERVEROUTPUT ON;

--         Table: PATIENT


CREATE TABLE PATIENT (
    patient_id    NUMBER(10)     CONSTRAINT pk_patient PRIMARY KEY,
    full_name     VARCHAR2(100) CONSTRAINT nn_patient_name NOT
       NULL,
    dob           DATE           NOT NULL,
```

```
    gender        CHAR(1)        CONSTRAINT chk_gender CHECK(gender
        IN ('M','F','O')),
    contact_no    VARCHAR2(15)   CONSTRAINT uq_patient_contact
       UNIQUE,
    blood_group   VARCHAR2(5),
    created_at    DATE           DEFAULT SYSDATE
);
```

Listing 1: Required Script Header Format

# 5 Submission Requirements

## 5.1 Required Deliverables

Submit a single compressed folder (`.zip`) named:
`DB_Lab_Project_GroupXX_Domain.zip` (e.g., `DB_Lab_Project_Group05_Hospital.zip`)

The folder must contain:

| # | File/Folder | Contents |
|---|---|---|
| 1 | `01_SRS.pdf` | Requirements document & data dictionary |
| 2 | `02_ERD_EERD.pdf` | ER and EER diagrams (clearly labeled) |
| 3 | `03_DDL.sql` | All CREATE TABLE, INDEX, VIEW statements |
| 4 | `04_DML_Inserts.sql` | All INSERT statements for data population |
| 5 | `05_Queries.sql` | All SELECT, JOIN, subquery, aggregate queries |
| 6 | `06_PLSQL.sql` | All procedures, functions, triggers, cursors, packages |
| 7 | `07_GUI/` | Complete GUI project source code |
| 8 | `08_Dashboard/` | Dashboard screenshots or source (if separate) |
| 9 | `09_Report.pdf` | Final project report (see Section 6.2) |
| 10 | `10_Demo_Video.mp4` | 5–10 minute walkthrough (optional but recommended) |

Table 5: Required Submission Files

## 5.2 Final Project Report Structure

The report (`09_Report.pdf`) must include:

1. **Title Page:** Project name, domain, group members, student IDs, date.
2. **Table of Contents**
3. **Introduction:** Domain description, problem statement, objectives.
4. **Requirements:** Functional requirements, data dictionary.
5. **Database Design:** ERD/EERD description, normalization justification.
6. **Schema Description:** Table descriptions and relationship explanations.
7. **Query Documentation:** Each major query with purpose, SQL code, and sample output screenshot.

8. **PL/SQL Documentation:** Each object with description, code listing, and test output.
9. **GUI Documentation:** Screenshots of each form with a brief description.
10. **Dashboard:** Screenshot and explanation of each metric.
11. **Challenges & Lessons Learned**
12. **Individual Contribution Table** (see Section 7).

# 6 Evaluation Rubric

**Total: 100 Marks**

## 6.1 Mark Distribution by Phase

| Phase | Component | Marks | LLO |
|---|---|---|---|
| 1 | Requirements Analysis & SRS | 10 | LLO-1 |
| 2A | ERD & EERD Design | 10 | LLO-2 |
| 2B | DDL Schema & Constraints | 10 | LLO-2 |
| 2C | Data Population (DML Inserts) | 5 | LLO-2 |
| 2D | Queries (SELECT, JOIN, Subquery, DCL) | 10 | LLO-2 |
| 3 | GUI Application | 20 | LLO-1 |
| 4 | PL/SQL (Procedures, Functions, Triggers, Cursors, Packages) | 25 | LLO-3 |
| 5 | Database Dashboard | 5 | LLO-1, LLO-2 |
| – | Viva / Presentation | 5 | LLO-4 |
| | **Total** | **100** | |

Table 6: Mark Distribution Across Phases

## 6.2 Detailed Rubric

| | |
|---|---|
| Complete SRS with 10+ functional requirements | 4 |
| Data dictionary for all entities (8+) | 3 |
| Non-functional requirements (3+) | 2 |
| Professional formatting and clarity | 1 |

| | |
|---|---|
| ERD with 8+ entities, correct notation | 3 |
| Minimum cardinalities met (1:M, M:M, 1:1) | 2 |
| Weak entity identified and modeled correctly | 1 |
| EERD with specialization hierarchy (2+ subtypes) | 2 |
| Aggregation/composition relationship in EERD | 1 |
| Diagram clarity and tool quality | 1 |

| | |
|---|---|
| 8+ tables with correct PK/FK definitions | 3 |
| 6+ foreign key relationships enforced | 2 |
| 3+ CHECK, 2+ UNIQUE constraints | 2 |
| 3NF normalization achieved and documented | 2 |
| 2+ indexes and 2+ views created | 1 |

| | |
|---|---|
| 5 correct SELECT with WHERE | 2 |
| 3 aggregate queries with GROUP BY | 2 |
| 3 subqueries (1 correlated) | 2 |
| 4 JOIN queries (multi-table included) | 2 |
| UPDATE, DELETE, DCL statements | 2 |

| | |
|---|---|
| Login screen with role-based access | 3 |
| 3+ CRUD forms with full validation | 9 |
| Multi-criteria search functionality | 2 |
| Data grid with auto-refresh | 2 |
| 2+ report generation (export) | 3 |
| Navigation and overall UX | 1 |

| | |
|---|---|
| 3+ stored procedures (with IN/OUT params, exceptions, nesting) | 8 |
| 2+ functions (one used in SQL query) | 4 |
| 3+ triggers (BEFORE/AFTER, meaningful logic) | 6 |
| 2+ cursors (explicit, parameterized) | 4 |
| 1 package with 2 procedures + 1 function + constant | 3 |

| | |
|---|---|
| 4+ KPI cards with live data | 2 |
| 2+ charts/graphs from DB queries | 2 |
| Summary table with aggregated data | 1 |

| | |
|---|---|
| Both members can explain all phases | 2 |
| Live demonstration runs without error | 2 |
| Ability to answer cross-questions | 1 |

# 7 Individual Contribution Declaration

Both members must include a **signed contribution table** in the final report. Marks for the viva will be awarded individually based on demonstrated understanding.

| Phase | Member 1 Contribution | Member 2 Contribution |
|---|---|---|
| Phase 1: SRS | | |
| Phase 2: ERD/EERD | | |
| Phase 2: DDL | | |
| Phase 2: Queries | | |
| Phase 3: GUI | | |
| Phase 4: PL/SQL | | |
| Phase 5: Dashboard | | |
| **Signatures** | | |

Table 7: Individual Contribution Declaration Table

**Important Note**

If a member cannot answer questions about their declared contributions during the viva, marks will be deducted at the instructor's discretion.

# 8 Timeline & Deadlines

| Week | Milestone | Deadline | Action Required |
|---|---|---|---|
| 3 | Group Formation | End of Week 13 | Submit group names & domain choice |
| 5 | Phase 1 | End of Week 14 | Submit SRS document |
| 7 | Phase 2A | End of Week 14 | Submit ERD & EERD |
| 9 | Phase 2B+2C | End of Week 15 | Submit DDL + INSERT scripts |
| 11 | Phase 2D | End of Week 15 | Submit Queries script |
| 13 | Phase 3 | End of Week 16 | Submit GUI source code |
| 14 | Phase 4 | End of Week 16 | Submit PL/SQL scripts |
| 15 | Phase 5 + Full Submission | End of Week 16 | Submit complete project zip |
| 16 | Viva / Presentation | Dead Week | In-lab demonstration |

Table 8: Project Timeline

**⚠ Important Note**

**Late Submission Policy:** A deduction of **25% per day** will apply to any late submission. No submission will be accepted after 3 days past the deadline without prior written approval from the instructor.

# 9 Academic Integrity Policy

1. All work must be original and created by the group members.
2. Sharing code between groups is strictly prohibited, even for reference.
3. Use of AI code generation tools (ChatGPT, Claude, Gemini, DeepSeek, Grok, GitHub Copilot, etc.) without disclosure is considered plagiarism.
4. Similarity between any two submissions will be investigated. If confirmed, **all involved groups receive zero**.
5. External libraries or frameworks must be cited in the report.

# 10 Quick Reference Checklist

Use this checklist before final submission to ensure completeness.

| Requirement | Min. Count |
|---|---|
| Entities documented in SRS | 8 |
| Attributes per entity (in data dictionary) | 5 each |
| Functional requirements listed | 10 |
| ERD relationships (1:M, M:M, 1:1) | 10 total |
| Weak entity in ERD | 1 |
| EERD with specialization hierarchy | 1 (2 subtypes) |
| Tables created with DDL | 8 |
| Foreign key relationships enforced | 6 |
| CHECK constraints | 3 |
| UNIQUE constraints | 2 |
| Rows per major table | 20 |
| Views created | 2 |
| Indexes created | 2 |
| SELECT queries | 5 |
| JOIN queries (incl. multi-table) | 4 |
| Subqueries (incl. correlated) | 3 |
| Aggregate queries | 3 |
| GUI: CRUD forms | 3 |
| GUI: Login with role-based access | 1 |
| GUI: Reports (exportable) | 2 |
| PL/SQL: Stored procedures | 3 |
| PL/SQL: Functions | 2 |
| PL/SQL: Triggers (BEFORE/AFTER) | 3 |
| PL/SQL: Explicit cursors | 2 |
| PL/SQL: Package | 1 |
| Dashboard KPI cards | 4 |
| Dashboard charts/graphs | 2 |
| Contribution table signed | Both members |

Table 9: Pre-Submission Checklist

# 11 Project Submission Protocols & Guidelines

To ensure a standardized evaluation process and to help you build a professional portfolio, all students must adhere strictly to the following guidelines. **Failure to comply with these protocols will result in penalties or disqualification from the Viva Voce.**

## 11.1 Group Composition & Submission Logic

- **Group Size:** Groups must consist of strictly **2 members**. No exceptions.
- **Designated Submitter:** Only one representative (**Group Lead**) from each group is required to upload the final project assets to Google Classroom.
- **File Naming Convention:** The compressed folder must be named using the roll numbers of all members.

**Required File Naming Format**

`21I-XXXX_21I-YYYY_DB_Lab_Project.zip`

Example: `21I-1234_21I-5678_DB_Lab_Project.zip`

## 11.2 Mandatory Deliverables (Google Classroom)

The Group Lead must upload a single `.zip` folder containing the following components:

### 11.2.1 A. Source Code

- All SQL and PL/SQL script files (`.sql`) organized in numbered order (e.g., `01_DDL.sql`, `02_DML.sql`, etc.).
- All GUI source files — include the complete project folder for the chosen technology (C#/.NET, Java, Python, etc.).
- A `README.txt` or `README.md` explaining:
  - How to set up the Oracle/SQL Server schema (which scripts to run in what order).
  - How to run the GUI application (IDE/runtime requirements, connection string setup).
  - Any external libraries or dependencies used.
- **Do NOT** include build artifacts, `.vs` folders, `bin/`, `obj/`, or `__pycache__` directories. Only source and necessary headers/assets.

### 11.2.2 B. Technical Report (PDF)

- A professionally formatted PDF report following the structure defined in Section 6.2.
- Must include screenshots of: all GUI forms, PL/SQL execution outputs, dashboard, and ERD/EERD.
- Must include a signed **Individual Contribution Table** (Section 7).

- Minimum report length: **25 pages** (excluding cover page and appendices).

#### 11.2.3 C. Video Demonstration

- A screen recording (`.mp4` preferred) of **5–10 minutes** duration.
- The video must include a **voice-over explanation** by group members walking through:
  1. Database schema and ERD walkthrough.
  2. Live execution of key SQL queries and PL/SQL blocks in Oracle SQL Developer.
  3. GUI: login, CRUD operations, search, and report generation.
  4. Dashboard with live data refresh.
- **Note:** A video showing only screen output without explanation will result in a grade deduction of up to 5 marks.

## 11.3 Individual Portfolio Requirements (Pre-Viva Eligibility)

**Individual Requirement — Both Members Must Complete**

This section is **individual**. Every student must complete these tasks independently to be eligible for the final Viva. To prepare you for the professional industry, every group member is required to publish their work online.

**Required Actions per Student:**

1. **GitHub Repository:**
   - Create a **public repository** and upload your project code.
   - Write a `README.md` that explains the project domain, setup instructions, technologies used, and your personal contribution.
   - Clearly document the PL/SQL components and schema structure in the README.
2. **LinkedIn Post:**
   - Share a short video snippet or screenshot of your GUI/dashboard in action.
   - Tag your teammate. Write about a technical challenge you solved (e.g., *"Finally figured out why my PL/SQL trigger wasn't firing on cascaded deletes..."*).
   - Mention your instructor in the post for added visibility.
3. **Medium Blog Article:**
   - Write a technical blog post of **300–500 words**.
   - Suggested topics: normalization decisions, a tricky PL/SQL problem, why you chose your GUI technology, or lessons learned from database design.

### 11.3.1 Verification Process

- A shared Google Sheet is linked to this assignment on Google Classroom.
- Every student must locate their name in the sheet and paste the **three URLs** (GitHub, LinkedIn, Medium) in the respective columns.
- The sheet URL: Click here to access the Verification Sheet

If a student **fails to populate the Google Sheet** with valid links to their individual posts, they will be marked **ineligible for the Viva Voce**. This will result in an **automatic Zero (0) for the entire project component**.

## 11.4 Academic Integrity (Zero Tolerance Policy)

1. **Plagiarism:** Copying SQL/PL/SQL code from online repositories (GeeksForGeeks, GitHub, Stack Overflow answers) or from other groups will lead to an **F grade (0 Marks)** for all involved parties. Automated similarity checks are run on all submitted scripts.
2. **AI Tool Usage Policy:**
   - You *may* use AI tools (ChatGPT, Copilot, etc.) to **understand concepts** — e.g., "Explain how a PL/SQL cursor works" or "What is the difference between BEFORE and AFTER triggers?"
   - You *may not* paste entirely AI-generated PL/SQL procedures, triggers, or GUI code into your submission without understanding and modifying it.
   - During the Viva, if you cannot explain a line of your own code (e.g., why `EXCEPTION WHEN NO_DATA_FOUND` is placed where it is), it will be treated as plagiarism and marked accordingly.
3. **Cross-Group Sharing:** Sharing any part of source code, SQL scripts, or report sections between groups is strictly prohibited, even "for reference."
4. **External Libraries:** Any third-party library or NuGet/pip package used must be cited in the report and the README.

# A Semester Project Self-Evaluation Form

**Instructions for Completing This Form**

Complete this form **honestly** before the Viva. Submit it as part of your `.zip` folder (scanned/signed PDF). The instructor uses this form to calibrate Viva questions to what you claim to have implemented. Inflated self-evaluations that do not match the demo will result in mark deductions.

## A. Group Details

**Section:** BSCS-4C **Session:** Spring 2025 **Lab Instructor:** Hassan Ahmed

| # | Student Name | Roll Number |
|---|---|---|
| 1. | _____ | _____ |
| 2. | _____ | _____ |

Group Member Details

**Project Domain / System Name:** _____

**GitHub Repository URL:** _____

## B. Module Implementation Checklist

Check the box **only** if the module is **fully functional**. If it works intermittently, crashes under edge cases, or only works on your specific machine, leave it unchecked and describe the issue in Section D.

### I. Database Schema & Design

**Entity-Relationship Diagram (ERD):** Complete ERD with 8+ entities and correct cardinalities.
**Enhanced ERD (EERD):** EERD includes specialization hierarchy with 2+ subtypes.
**DDL Schema:** All tables created with PK, FK, CHECK, and UNIQUE constraints.

**Normalization:** All tables satisfy 3NF — documented and justified in report.

**Views & Indexes:** At least 2 views and 2 indexes created.

### II. Data & Query Operations

**Data Population:** 20+ rows per major table with realistic domain-consistent data.

**SELECT Queries:** 5+ SELECT queries with WHERE filters.

**Aggregate Queries:** 3+ GROUP BY queries using COUNT, SUM, AVG, MIN, or MAX.

**JOIN Queries:** 4+ JOINs including at least one multi-table (3+ tables) join.

**Subqueries:** 3+ subqueries including at least 1 correlated subquery.

**DCL:** GRANT and REVOKE demonstrated on at least one database object.

## III. PL/SQL Procedural Programming

**Stored Procedures (3+):** Includes IN/OUT parameters, exception handling, and nested calls.

**Functions (2+):** At least one function is called inside a SQL SELECT statement.

**BEFORE Trigger:** Auto-generates ID or sets default values before insert.

**AFTER UPDATE Trigger:** Performs audit logging or cascaded business logic.

**AFTER DELETE Trigger:** Archives record or logs deletion event.

**Explicit Cursor (2+):** Uses OPEN/FETCH/CLOSE; at least one is parameterized.

**Package (1):** Contains 2+ procedures, 1+ function, 1 package-level constant/variable.

**Anonymous PL/SQL Blocks (2+):** Demonstrates IF/LOOP/EXCEPTION and DBMS_OUTPUT.

## IV. GUI Application

**Login Screen:** Authentication with at least 2 roles (Admin / User).

**CRUD Forms (3+ entities):** Add, Update, Delete, and Search fully functional with input validation.

**DataGrid Display:** All records displayed; auto-refreshes after every operation.

**Multi-Criteria Search:** Filter by 2+ fields simultaneously.

**Report Generation (2+):** Exportable reports (PDF/Excel/CSV) with title, date, and aggregates.

## V. Database Dashboard

**KPI Cards (4+):** Displays live aggregate values fetched from the database.

**Charts/Graphs (2+):** Bar, pie, or line chart populated from real queries.

**Summary Table:** Grouped/aggregated view of key domain metrics.

**Real-Time Refresh:** Dashboard reloads from DB on every visit or button click.

## VI. Advanced / Bonus Features (Optional)

Dashboard date-range filter for metrics.

Dashboard export to PDF or image.

Pagination for large datasets in GUI.

Role-based dashboard (Admin sees all; User sees own data).

## C. Individual Contributions

Identify who built what. Be specific. Vague entries like "I helped" are not acceptable.

**Member 1:** _____ **Roll No.:** _____

Specific modules/tasks implemented:

**Member 2:** _____ **Roll No.:** _____

Specific modules/tasks implemented:

## D. Reflection & Analysis

**1. The "Normalization & Design" Challenge:**
*(How did you decide on your entities? Did you encounter redundancy issues during design? How did you resolve them?)*

**2. PL/SQL Trade-offs:**
*(Why did you use a trigger instead of a procedure for a particular operation? What was the trade-off? Did you encounter mutating table errors?)*

**3. Known Bugs / Unfinished Modules:**
*(Be honest. Does a specific form crash on empty input? Is a trigger not firing correctly in all cases?)*

## E. Individual Portfolio Links

| Platform | Member 1 URL | Member 2 URL |
|---|---|---|
| GitHub | | |
| LinkedIn | | |
| Medium Blog | | |

Portfolio Links (must also be entered in the shared Google Sheet)

## F. Signatures & Integrity Declaration

By signing below, we certify that **all work in this project was created by us**. We understand that code generated entirely by AI without understanding, or copied from any online source or another group, constitutes academic dishonesty and will result in **disqualification from the Viva and zero marks** for the project.

Member 1 Signature
Name: _____
Date: _____

Member 2 Signature
Name: _____
Date: _____